Document Number: P2352R0
Date: 2021-04-04
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24

Summaries of SG16 meetings are maintained at https://github.com/sg16-unicode/sg16-meetings. This paper contains a snapshot of select meeting summaries from that repository.

- December 9th, 2020
- January 13th, 2021
- January 27th, 2021
- February 10th, 2021
- February 24th, 2021
- March 10th, 2021
- March 24th, 2021

Previously published SG16 meeting summary papers:

- P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25
- P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20
- P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03
- P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09
- P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22
- P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25
- P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11
- P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27
- P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26
- P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11

December 9th, 2020

Draft agenda:

- P2093R2: Formatted output:
  - Continue discussion.

Attendees:

- Hubert Tong
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Tom provided some administrative updates:
  - A reminder that SG16 telecons operate under the WG21 code of conduct as described in the "Code of conduct" section of WG21 Standing Document #4 and, by extension, the ISO Code of Conduct and IEC Code of Conduct.
  - A reminder that SG16 is a public group and that minutes of SG16 telecons are made publicly available. Participants may request that sensitive information not be minuted.
  - A draft paper calling for the creation of a WG21 managed chat service will be submitted for the December 15th mailing.
  - *[Editor's note: the submitted paper is P2263R0: A call for a WG21 managed chat service. An administrative telecon will be scheduled in early January to discuss it with the intent to have a draft revision addressing any feedback prepared for the February pre-meeting administrative telecon.]*
  - The deadline for initial proposals of new features to be included in C2x is 2021-08-27.
  - The next C2x meeting is scheduled for March 8-12, 2021.
  - Tom asked to clarify the next steps following our recent discussion of P2194R0 and asked who is planning to write the paper to re-word translation phase 1 in terms of Unicode scalar values rather than basic source characters and universal-character-names (UCNs).
  - Jens stated he will write that paper, but noted some complications:
    - Not all string literals should be transcoded into the execution encoding because literals that appear in contexts such as `static_assert` and language linkage are evaluated at compile-time and have different encoding implications.
    - The issue of translation phase 5/6 confusion regarding string literal concatenation and conversion to the execution encoding remains.
    - It may become necessary to move string literal conversions and concatenation to translation phase 7.
    - Care will be required to ensure concatenation of string literals does not result in construction or extension of escape sequences.
- P2093R2: Formatted output:

- PBrett provided an introduction:
  - It has been a month since we last reviewed.
  - There has been some good discussion on the mailing list.
  - Victor will continue his presentation.
- Victor presented:
  - The intent of the proposal is to integrate formatting facilities with output streams.
  - A survey of current Java, Python 3, and Rust releases was conducted to ascertain their behavior on a Russian Windows system (Active Code Page (ACP) set to Windows-1251; console encoding set to CP866) when a string containing Russion and Greek characters is written to stdout with stdout directed to a console and again with stdout redirected to a file.
    - For Java, `java.lang.System.out` was used.
    - For Python, `print` was used.
    - for Rust, `std::print` was used.
    - For each language, redirection of stdout to a file was done using the `cmd.exe` shell.
  - Java failed to display the correct characters to the Windows console; UTF-8 output was produced when stdout was redirected to a file.
  - *[Editor's note: See further discussion below regarding the Java behavior.]*
  - Python threw an exception when trying to convert the Greek characters to Windows-1251.
  - Rust displayed the correct characters to the console and UTF-8 output was produced when stdout was redirected to a file.
- PBrett noted a difference from C++ that each of these languages shares; they each always use a UTF encoding for strings.
- PBrett added that both Python and Rust appear to behave in an arguably correct manner.
- PBrett stated that he has many tools that operate in ISO-8859 encodings and where generation of UTF-8 output would not produce the expected behavior.
- PBrett noted that this is a new facility, so it could behave differently.
- Tom expressed surprise that the Java test produced UTF-8 when stdout was directed to a file as that does not match his understanding of Java's behavior.
- Steve noted that such surprising behavior can be the result of mismatched encoding expectations and asked if the source file encoding was correct.
- Steve added that this is an easy thing to get wrong.
- Victor replied that all source files were UTF-8 encoded.
- Tom stated that the Java compiler uses the locale to determine souce file encoding unless invoked with the `-encoding` option; similar to how Microsoft Visual C++ behaves.
- Zach asked if source file encoding could influence the encoding used for file redirection.
- Victor replied that he would conduct additional tests.

- *[Editor's note: later tests revealed that the reported behavior for Java was incorrect. The Java compiler had been invoked without a `-encoding` option, so the UTF-8 encoded source file was misinterpreted as being Windows-1251 encoded and the compiler converted string literals from Windows-1251 to UTF-16. When the strings were then written via `java.lang.System.out`, the prior conversion was reversed when `java.io.PrintStream` converted the string to print from UTF-16 back to Windows-1251. The result was that the original bytes from the source file encoding of the string literal were written to stdout. This produced mojibake on the Windows console and gave the appearance that the program had (intentionally) generated UTF-8 in the file redirection scenario. Note that all valid UTF-8 code unit sequences are also valid Windows-1251 code unit sequences.]*
- Tom asked Victor if he could extend his testing to cover cases where stdout was redirected to a pipe that was connected to stdin of another process.
- Victor replied that he would look into that.
- PBrett recalled that, in prior discussion, we had started discussing the association of the execution encoding and run-time encoding and how that influences behavior.
- Victor summarized the proposed behavior and some of the prior discussion:
  - Decisions are based solely on execution encoding (known at compile time).
  - Use of locale settings would be challenging due to the involvement of multiple encodings.
  - The encoding of the format string must be consistent.
  - Tom had raised the question of a z/OS implementation using UTF-8 as the execution encoding, but operating in an EBCDIC environment.
- PBrett observed that use of Microsoft's `/utf-8` option would cause UTF-8 output to be generated and asked about what should happen when that option isn't used.
- Victor replied that the behavior depends on the system configuration and that the proposal specifies that bytes be passed through unmodified in that case.
- Steve relayed experience with tools that end up writing ANSI terminal escape sequences into a file when output is redirected and noted the difficulty that would be encountered when attempting to determine what kind of device receives the final output; examples may involve multiple ssh hops.
- Tom opined that the only case he is aware of where writing directly to the device instead of to the file stream makes sense is on Windows where it can be known definitively that the file stream is attached to a console.
- Hubert explained that, on z/OS, an application can internally be in ASCII or EBCDIC mode, open file handles can be imbued with the property of being ASCII or EBCDIC, and the C-level I/O APIs can automatically translate between them for, at least, single-byte encodings.
- Jens commented that the proposed feature appears to be centered around a special facility for Windows and expressed uncertain skepticism regarding driving a design around it.
- Jens added that, in the z/OS scenario as Hubert described it, there appears to be uncertainty that the facility would handle variable length encodings adequately.

- Jens stated that, given the wide array of platforms supported by the C++ standard, that he would prefer to craft a design around an abstract console stream.
- PBrett expressed concern that this facility might not be used much on Windows because use of Microsoft's `/utf-8` option is not common.
- Victor disagreed with the notion that the design is centered around Windows and a particular corner case; the goal is to fix the general problem of producing correct output.
- Victor stated that, with respect to use of the `/utf-8` option, that he is open to the changes Tom suggested to transcode as necessary and write directly to the console when output is directed there regardless of what the execution encoding is.
- Tom agreed with Victor that the concerns are more fundamental and not a special case; the goal is to improve support for an internal encoding distinct from the external encoding.
- Tom asked about coexistence with other formatting facilities and what concerns arise due to potentially divergent behavior; programs won't be rewritten over night to migrate all `printf()` uses to `std::print()`.
- Zach asked what the tradeoffs in design options are, what gets broken based on design choices, and how much of this can be left implementation-defined.
- Zach expressed support for the approach described in the paper.
- Hubert stated that encompasing the console in a separate facility would pose challenges because it assumes the presence of a unique "console" in the environment.
- Hubert added that an important concern is encoding of string literals vs encoding of strings received from the environment. For example, regex libraries tend to use a possibly pre-compiled pattern encoded in the execution encoding, but operate on strings provided by the environment; it is necessary to differentiate these.
- PBrett agreed with Hubert's concerns.
- Steve stated that the proposed feature is at least partly QoI and that, if we can specify this with sufficient latitude for Microsoft to make their customers happy, great; the Windows console capabilities are not portable.
- Steve added that locale information is input to the program and used to interpret bytes received as input.
- Steve raised the concern that the proposed approach may enclose transcoding behavior too deeply where it can't be fixed if there is a problem.
- Tom agreed with Steve's concern and noted parallels with Jens' previous request to separate transcoding features.
- PBrett asked how encoding errors should be handled.
- Victor replied that the current implementation throws an exception, but that there has been a recommendation to use U+FFFD character substitution instead.
- Zach noted that use of subtitution characters matches Unicode recommendations.
- Victor noted that, when writing to the console, substitution is safe since it is known that the output would be incorrect anyway.
- PBrett acknowledged that perception; the text gets converted to photons.

    - Victor added that the Console font is typically limited in what can be displayed as well.
    - PBrett asked if there were any objections to use of substitution characters in error handling scenarios.
    - No objections were raised.
    - Jens stated that the proposed feature is effectively a large hammer being aimed at more nails than is really desired. For example, in safety critical scenarios, a programmer may need to be notified of errors; displaying a Unicode replacement character to an aircraft pilot is less than helpful.
    - Jens reflected that, on the other hand, perhaps this is a tool intended for common users where such substitutions are not an issue.
    - Jens expressed support for transcoding operations being explicit at program boundaries and noted that there are a number of encodings involved; execution encoding, locale dependent input, locale dependent output (potentially multiple; e.g., Windows ACP and console encoding).
    - Jens opined that we should promote a programming model that puts more control in the hands of the programmer.
    - Victor replied that `std::format()` doesn't do any transcoding at present; that the encodings must match and that programmers must get data into the right encoding first.
    - Hubert noted the law of unintended consequences; that trying to ensure a behavior in one case can result in undesired behavior in another.
    - Steve agreed and noted that use of `isatty()` tends to be problematic as it can be surprising when behavior changes based on redirection.
    - PBrett asked Tom what polls should be conducted.
    - Tom responded that he did not think discussion had progressed to a point where we all hold well-informed positions.
    - Tom stated that the additional work Victor has agreed to do should help strengthen our understanding and positions; we should therefore wait for this additional information before conducting any polling.
- Tom stated that the next meeting will be on January 13th and the agenda will include:
    - P2246R0: Character encoding of diagnostic text
    - SG16, SG22, and WG14 coordination; we'll discuss how to make forward progress on SG16 issues labeled with the WG14 tag.
    - WG14 N2620: Restartable and Non-Restartable Functions for Efficient Character Conversions | r4

January 13th, 2021

Draft agenda:

- P2246R0: Character encoding of diagnostic text

    - And companion paper: WG14 N2563: Character encoding of diagnostic text
- SG16, SG22, and WG14 coordination.
    - Priority and owners for SG16's open WG14 issues.
- WG14 N2620: Restartable and Non-Restartable Functions for Efficient Character Conversions | r4

Attendees:

- Aaron Ballman
- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Tom stated that his paper to clarify guidance regarding use of BOMs in UTF-8 text was submitted to the UTC and given paper number L2/21-038.
    - The submitted paper is available at https://www.unicode.org/L2/L2021/21038-bom-guidance.pdf.
- P2246R0: Character encoding of diagnostic text:
    - Aaron provided an introduction:
        - The basic problem is that we have features that require source code fragments to be reproduced in diagnostics without clear specification or limits regarding how that should be done.
        - Depending on the various encodings in use, accurate representation of the original source code may not be possible.
        - The degree to which these fragments are preserved and reproduced should be a QoI issue.
        - WG14 approved the C version of this paper (WG14 N2563: Character encoding of diagnostic text).
        - The wording changes for the C standard have a little additional cleanup; some uses of "may" were changed to "should".
        - A goal is to keep the C and C++ standards aligned.
    - PBrett asked if the proposed changes might be considered editorial.
    - Hubert opined that considering them editorial would be contentious.
    - Aaron responded that the changes should not be considered editorial.

- PBrett asked if there is anything evolutionary about the changes.
- Aaron replied that he didn't think so, but that EWG participants may have a different perspective; for example, a desire to define a character set for diagnostics, though he would be surprised.
- Steve noted that we have parallel work in progress to specify translation within the standard in terms of a Unicode encoding.
- Hubert observed that the standard currently specifies different requirements for the `static_assert` declaration and `#error` directive vs the `[[deprecated]]` and `[[nodiscard]]` attributes; the former require the message to reproduce the text while the latter specify normative encouragement.
- Hubert noted that the status quo is not that diagnostics are QoI; there are requirements on the text produced, so changes could impact implementations.
- Hubert observed that the `#error` directive requires reproducing preprocessing tokens, potentially including string literals, at an earlier phase of translation than, for example, `static_assert`.
- Hubert requested that the paper clarify how diagnostics produced at different translation phases be handled.
- Aaron stated that he was not aware of a requirement that diagnostics be textual; an implementation that presented a frowny face or a graphical image of the source code would be conforming.
- PBindels chimed in from chat to state that Evoke emits emoji for its error state and that he felt called out.
- Victor also chimed in from chat to suggest to PBindels that emoji is still text and that Evoke should emit a gif with a relevant meme.
- Hubert responded that there is a requirement if the standard states one; the implementation must have some means to present a message.
- Aaron asked if a change to the prose was desired.
- Hubert responded affirmatively; that he would like to see a change of the rationale to clarify that there is no increase in requirement for some representation of the text in comparison to the status quo.
- Aaron agreed to make such a change.
- Corentin recalled earlier discussions regarding translation phase 1, string literals always being in execution encoding, and whether `static_assert` might require special handling.
- Corentin expressed contentedness with the paper as presented and stated that the standard should not have to specify how a string literal is presented.
- Aaron agreed with one exception; `#error` is specified solely in terms of preprocessing tokens.
- Corentin acknowledged; a string literal is a token in that case and `#error` processing occurs before conversion to the execution character set in translation phase 5.
- Hubert noted that the status quo is that preprocessing tokens are limited to the basic source character set due to translation phase 1 converting other characters to UCNs.

    - Hubert added that we have direction that might change this, but that this paper should not be held up because of that; Jens' effort may need to consider this though.
    - Hubert observed that the standard does not specify how an implementation represents text in the execution character set, but that it needs to acknowledge that a translation is required when producing a diagnostic.
    - Aaron asked if some string literals, such as those used in attributes, should even undergo translation to the execution character set.
    - PBrett replied that an implementation performs conversion to the execution character set and therefore has the means to undo the conversion.
    - Steve noted that an implementation presumably will retain access to the original source code; a reverse map to the original text suffices in the worst case.
    - Hubert stated that it isn't necessarily true that a compiler can easily map from a converted string literal back to the original text after translation phase 5.
    - Mark noted that, with regard to attributes, they can include arbitrary expressions, not just string literals.
    - Corentin stated that this discussion is expanding the scope of what is required and repeated his opinion of the paper being acceptable as presented.
    - Corentin added that, with respect to string literals, that programmers be able to rely on them being consistently converted; consistency is important for reflection and other compile-time programming facilities.
    - Steve opined that some of this discussion is well in to QoI territory; if a compiler can't display source code in a sensible manner to the programmer, then all bets are off.
    - Hubert stated that he would be happy with changing the current uses of "shall" to "should" via this paper, but that it isn't strictly necessary at this time; we don't want to exclude any execution environments.
    - Aaron agreed.
    - Hubert added that the wording for `#error` should be changed to match.
    - **Poll: When diagnostic messages quote a portion of source code, the preservation of text semantics is merely a quality of implementation concern.**
        - **Attendance: 11**
        - **SF F N A SA**
          5 2 2 0 0
        - **Consensus is in favor.**
    - **Poll: We agree with removing the specific character set from static_assert along the lines of P2246.**
        - **Attendance: 11**
        - **No objections to unanimous consent.**
    - **Poll: Forward P2246 to EWG**
        - **Attendance: 11**
        - **No objections to unanimous consent.**
- SG16, SG22, and WG14 coordination:
    - Aaron introduced:
        - SG22 will have its first meeting soon.

        - Any WG21 papers targeted to SG22 will be forwarded to WG14 and SG22 will evaluate whether the paper is appropriate for both groups. Authors will then submit papers to WG14.
        - SG22 will assist with finding people to present at WG14 meetings on behalf of WG21 authors that are unable to attend a WG14 meeting.
        - WG14 does not currently have a study group that focuses on text issues.
        - WG14 participants tend to be passionate about character sets.
- Review of SG16's open WG14 issues:
    - Issue #62: WG14 N2594: Disallow mixed string literal concatenation:
        - JeanHeyd reported that WG14 N2594 was adopted for C2x and that this issue can be closed.
        - JeanHeyd noted that there was a request for a follow up paper to allow mixed literals with defined semantics.
        - Aaron added that the sdcc compiler implements mixed string literals with useful semantics.
    - Issue #56: WG14: Improve support for Unicode characters in identifiers:
        - Steve volunteered to look at this.
        - Tom commented that WG14 generally likes to see two implementations before adopting a feature and that a change to the C++ standard generally counts as one.
        - Aaron confirmed and noted that more implementations increases the motivation for standarizing existing behavior.
        - Corentin asked if compatibility with C++ is considered good motivation.
        - Aaron replied that it is seen more as weak motivation.
    - Issue #55: WG14: Named universal character escapes:
        - Tom stated that it is still on his plate to get a revision of P2071 submitted to WG21.
        - PBrett suggested that SG22 be included on the next revision of the paper.
        - Tom agreed.
    - Issue #54: WG14: Make char16_t/char32_t string literals be UTF-16/32:
        - Tom noted that this is standardizing existing practice.
        - JeanHeyd volunteered to own this.
        - PBrett volunteered as backup if JeanHeyd becomes too busy with his other efforts.
    - Issue #5: WG14 N2231: char8_t: A type for UTF-8 characters and strings:
        - Tom stated that he is actively working on this.
- WG14 N2620: Restartable and Non-Restartable Functions for Efficient Character Conversions | r4:
    - Tom apologized to JeanHeyd, but stated that discussion of this paper would have to wait until the next telecon.
- Tom stated that the next telecon will be held January 27th. The agenda will include a presentation and discussion of Jonathan Müller's Lexy parser combinator library and, of course, JeanHeyd's WG14 N2620 postponed from today's meeting.

January 27th, 2021

Draft agenda:

- Presentation and discussion with Jonathan Müller regarding his lexy parser combinator library (Tutorial, Reference), and the text and Unicode related challenged he faced, how he solved them, and what C++ standard language or library features he would have benefitted from.
- WG14 N2620: Restartable and Non-Restartable Functions for Efficient Character Conversions | r4

Attendees:

- Corentin Jabot
- Daniela Engert
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Jonathan Müller
- Mark Zeren
- Peter Brett
- Steve Downey
- Tomasz Kamiński
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Presentation on lexy's text and Unicode challenges by Jonathan Müller:
  - Jonathan's presentation slides are avilable here.
  - Part 1: Inputs and Encodings:
    - Jonathan presented:
      - Several classes are provided to handle input via ranges, strings, and buffers, each of which provides a reader class.
      - The reader class provides access to the input data independent of the specific input class.
      - Encoding classes provided for each of the supported encodings define traits and operations for an encoding.
      - The encoding classes are superficially similar to `std::char_traits`; they define a character type, an integer type to be used as an EOF sentinel,

conversion operations from the character type to the integer type, secondary types that may be used with the encoding, and encode operations.
    - The supported encodings are:
        - Default: uses `char` as the character type, `int` as the integer type, and `-1` as the EOF value.
        - Raw: uses `unsigned char` as the character type, `int` as the integer type, and `-1` as the EOF value. `char` and `std::byte` can be used as secondary character types.
        - ASCII: uses `char` as the character type, `char` as the integer type, and `0xff` as the EOF value.
        - UTF-8: uses `char8_t` as the character type, `char8_t` as the integer type, and `0xff` as the EOF value. `char` can be used as a secondary character type.
        - UTF-16: uses `char16_t` as the character type, `std::int_least32_t` as the integer type, and `-1` as the EOF value. `wchar_t` can be used as a secondary character type if it has the same size as `char16_t`.
        - UTF-32: uses `char32_t` as the character type, `char32_t` as the integer type, and `0xffffffff` as the EOF value. `wchar_t` can be used as a secondary character type if it has the same size as `char32_t`.
    - Wish list:
        - The ability to determine the encoding of ordinary string literals.
        - The ability to convert between the original character types (`char`, `wchar_t`) and the newer types (`char8_t`, `char16_t`, `char32_t`) without the spectre of undefined behavior.
        - A library function to heuristically determine or guess the encoding for some input.
- *[Editor's note: Right at the start of Jonathan's presentation, one of the editor's sons showed up bleeding from having fallen while climbing over a fence. Recovery was quick, and the editor was appreciative of the excellent slides that enabled him to fill in what he missed of the presentation.]*
- Hubert noted that use of `0xFF` as the EOF value for the ASCII encoding could be problematic since `char` may be a signed type, but is probably ok if it is just an internal implementation detail.
- PBrett observed that, for encodings with a larger integer type, buffer space may not be used efficiently.
- Jonathan replied that buffers always store values in the character type, not the integer type.
- Victor asked how ill-formed input that might have a character value that matches the EOF value is handled.
- Jonathan replied that processing of such input will end prematurely.

        - PBrett stated that it seems reasonable for the library to require well-formed input.
        - PBrett asked if a library solution that provides a "blessed" cast operation for handling input in the secondary character types would suffice.
        - Jonathan replied that it would.
        - Corentin mentioned having discussed such a possibility with Richard Smith in the past, particularly with regard to the possibility of `constexpr` support.
        - Jonathan confirmed that `constexpr` support would be useful.
        - Steve stated that there is a need for that feature for historical interfaces that have `char*` parameters.
        - Jonathan acknowledged the need and explained that these interfaces accept either the primary or secondary type and use `reinterpret_cast` internally.
        - Hubert asked if these conversions are needed only in one direction or in both.
        - Jonathan replied that they are one directional; the user will not modify the text after handing it off and the library does not hold on to input beyond a call.
        - Jonathan added that, if a buffer is provided, the data is copied.
        - Jens explained that the reason that the `reinterpret_cast` results in undefined behavior is because the compiler can assume no aliasing of most types.
        - Corentin asked if a bless function could work at `constexpr` time.
        - Hubert replied that this is different than the `std::bless` that has been discussed in the past; to bless means to start the lifetime of an object.
        - Jens stated that there are two approaches we can consider:
            - Changing the alias rules and thereby prohibit related optimizations.
            - Extending the memory model in some way to enable such magic.
        - Mark noted that the anti-aliasing features of `char8_t` were a motivator for adopting the type.
        - Jens added that this is a research opportunity.
        - Jonathan obseved that undefined behavior seems to be ok for users for now.
        - Zach stated that he has also written a parser combinator library, but did so in a way that did not require use of `reinterpret_cast`.
        - Corentin noted from chat that P1885 exposes the encoding of ordinary string literals.
    - Part 2: Unicode-Aware Rules:
        - Jonathan presented:
            - Parsing requires converting character input to integer input to check for EOF.
            - Comparing input to literal values also requires conversions.
            - The allowed conversions depend on the encoding in use.
            - Transcoding support is limited to conversion from ASCII to other encodings and assumes that the code point value in the target encoding matches the ASCII character code and is representable with a single code unit.

            - Matching of literal values is restricted to ASCII unless the literal has a `char8_t`, `char16_t`, or `char32_t` based type or if an explicit encoding is specified.
            - A rule is provided for matching a Unicode BOM.
            - A rule is provided for reading a code point for encodings other than default and raw.
            - Input is assumed to be well formed; no validation is performed.
            - Wish list:
                - Unicode character classification functions.
                - A `std::code_point` type.
                - Support for validating encoded input.
            - Features not actually needed for this part:
                - Decoding facilities.
                - Transcoding facilities.
        - *[Editor's note: The editor's internet connection decided to take some time off just as Jonathan started presenting part 2. It came back fairly quickly, but a few minutes of presentation were lost. Gaps were again filled in thanks to the excellent slides.]*
        - Hubert asked what features would be desirable in a `std::code_point` type.
        - Jonathan presented lexy's `code_point` type and its `is_valid()`, `is_surrogate()`, `is_scalar()`, `is_ascii()`, and `is_bmp()` members.
        - PBrett observed that the desire is for something like Unicode properties.
        - Jens noted the much simpler requirements.
        - PBrett asked if higher level features are provided that could handle Unicode normalization.
        - Jonathan replied that there are not currently, but that adding such support seems feasible.
        - Zach noted that there are normalization equivalences, but also equivalence for collation purposes and that a lot of effort is required to get that working.
        - Corentin noted from chat that P1628 provides support for Unicode character classification and that an implementation is available at https://github.com/cor3ntin/ext-unicode-db/blob/master/generated_includes/cedilla/properties.hpp.
    - Part 3: File Input:
        - Jonathan presented:
            - Buffers have an associated encoding.
            - Endian concerns, including handling of BOMs, are addressed when filling a buffer.
            - A function is provided for populating a buffer from a file.
            - File reading is done in binary mode and without encoding validation.
            - Wish list:
                - `std::read_file()`.
                - Endian conversion facilities.

            - Facilities to heuristically identify the encoding of arbitrary input.
        - Tom noticed that, if a BOM is not present, that big endian is assumed and asked if the default shouldn't be the native endian order for data in memory.
        - PBrett agreed that, for data in memory, yes, but for data at rest, big endian is the default.
        - Jonathan explaind that BOM assumptions are only used for files at present, so assuming big endian seems correct.
        - PBrett agreed that a `std::read_file()` function would be useful.
        - Jens stated in chat that support for endian conversions is in progress; P1272R3.
    - Part 4: The `as_string` callback:
        - Jonathan presented:
            - When a production rule is matched, the parsed lexeme is passed to a functor that can receive it as a string-like type, a C string, or as a lexeme with associated reader class.
            - A lexeme is a range over the input.
            - Wish list:
                - Encoding facilities.
                - Transcoding facilities.
    - PBrett noted that Zach's Boost.text library does some of this.
    - Zach agreed and noted support for transcoding.
    - PBrett asked if support for grapheme rules had been considered.
    - Jonathan replied that he has considered such rules, but that such support requires the Unicode DB.
    - Mark asked if grapheme interfaces were available in the standard, whether they would be used.
    - Jonathan replied, yes and stated that he intends to implement more Unicode rules, but they are work.
    - Tom summarized one of his big take aways from the presentation is how encodings were handled; that support was limited to ASCII unless there was evidence in the type system for a more capable encoding.
    - PBrett stated that one of his big take aways is that a number of the items in the wish lists are already in the pipeline.
    - Jens agreed and noted that these are some of the easier items to address.
    - Jens noted that there would be clear benefit from access to Unicode DB character properties.
    - PBrett asked if Jonathan would make the slides available.
    - Jonathan agreed to do so.
    - *[Editor's note: Jonathan did so and they are linked above.]*
    - Corentin stated that the ability to provide conversions between `char` and `char8_t` is something that we might be able to do for C++23.
    - PBrett suggested that one way to do that might be to take a span and return another span.

    - Steve agreed that the ability to perform such conversions would be useful, but wondered if it can be done without losing aliasing benefits.
    - Jens suggested postponing how to alias `char` and `char8_t` until a proposal is provided.
    - Zach commented that he and Jonathan made many similar choices in their implementations, but noted one point of difference; Zach tried to deduce the encoding all the time.
    - JeanHeyd observed that, with Corentin's P1885, the literal encoding will be known.
    - JeanHeyd added that gcc trunk already has support for the predefined macro and that Clang and MSVC are making progress.
- WG14 N2620: Restartable and Non-Restartable Functions for Efficient Character Conversions | r4>:
    - JeanHeyd presented:
        - The paper was presented to WG14 in November.
        - WG14 requested that additional conversion functions be provided to convert between UTF encodings.
        - The paper proposes conversion functions from the locale sensitive MB/wide encodings to UTF encodings.
        - Converters are provided that perform per-code-unit translation and per-string translation.
        - These functions avoid the historical issues with variable length encodings.
        - MAX macros are provided to avoid having to check for and handle insufficient output errors.
        - These functions differ from `iconv` because C doesn't have a locale or encoding tag suitable for specifying an encoding.
    - PBrett asked to verify that sizes are always expressed in code units in the paper.
    - JeanHeyd replied that they are.
    - PBrett observed that the historical designs returned a positive value to indicate the number of code units that were written.
    - JeanHeyd acknowledged and explained that a different design is proposed here because of ambiguities that arise with a return value of 0.
    - Steve asked how the number of code units that were consumed and written are returned.
    - JeanHeyd replied that the provided pointers are updated, so theu caller can calculate.
    - Jens observed that the proposed interface always requires two modifications when some input is consumed and output is produced; an alternative would be a range where only the input is bumped.
    - JeanHeyd replied that convenience functions that provide that simpler interface could potentially be provided.
    - Jens asked for more explanation for the use of a pointer/size pair as opposed to a pointer/pointer pair.
    - JeanHeyd replied that null can be passed for the size.

    - Jens expressed concerns about security issues.
    - Tom stated that a null could be passed for an end pointer to accomplish the same goals; and the same security concerns.
    - PBrett suggested postponing further discussion until the next meeting.
- Tom stated that the next meeting will be February 10th and that WG14 N2620 will be top of the list.
- Tom thanked Jens for his recent draft paper proposing changes to the UCN model and that he would put that paper on the agenda soon.
- Tom added that he is thinking about dedicating an upcoming telecon to discussion of what we want to try and complete for C++23.

February 10th, 2021

Draft agenda:

- WG14 N2620: Restartable and Non-Restartable Functions for Efficient Character Conversions | r4:
    - Continue discussion started at the last telecon and in recent email discussion.
- P2093R3: Formatted output:
    - Review Victor's updates since our review of P2093R2 on 2020-12-09.

Attendees:

- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- WG14 N2620: Restartable and Non-Restartable Functions for Efficient Character Conversions | r4:
    - JeanHeyd stated that he has not yet completed the benchmarks requested in the email discussion.

- JeanHeyd shared code and demonstrated three possible signatures for the conversion functions:
  - 1) The form proposed in the paper.
    - `mcerr_t(const charX** ibegin, size_t* isize, charY** obegin, size_t* osize)`
    - Con: Each call requires writes to `*ibegin`, `*isize`, `*obegin`, and `*osize`.
  - 2) Iterator pairs passed by reference.
    - `mcerr_t(const charX** ibegin, const charX** iend, charY** obegin, charY** oend)`
    - Pro: Each call only requires writes to `*ibegin` and `*obegin`.
  - 3) Iterator pairs, begin passed by reference, end passed by value.
    - `mcerr_t(const charX** ibegin, const charX* iend, charY** obegin, charY* oend)`
    - Pro: Each call only requires writes to `*ibegin` and `*obegin`.
    - Con: No support for unbounded reads and writes.
- Zach asked if, for #3, both `ibegin` and `iend` are required to be non-null.
- JeanHeyd replied that they are.
- Zach stated that, if null is passed for `oend`, that should allow unbounded writes, and if null is passed for `obegin`, that should allow counting code units.
- JeanHeyd acknowledged and added that passing null for both would support counting and validation.
- Tom noted the assumption that the count is returned via the return value in that case.
- JeanHeyd agreed and noted that would require special error return values ike `(size_t)-3`.
- PBrett asked if named constants would be provided for such error values.
- JeanHeyd replied yes and that they already exist.
- Jens requested that the paper clearly illustrate the three modes of operation (counting, validation, and conversion) and provide examples of each.
- Jens noted that `iconv` does not support unbounded buffers.
- Jens added that support for unbounded buffers would be novel and carries significant risk.
- Jens observed that the arguments in the signatures presented differ from what is proposed in the paper.
- JeanHeyd acknowledged and noted there is a lack of consistency in existing interfaces.
- Jens suggested letting WG14 choose the argument order.
- Tom noted that returning `size_t` with special error values makes for verbose call sites since there is no simple way to test for an error.
- JeanHeyd agreed and noted that the proposed signature was designed to avoid that issue.
- JeanHeyd added that a caller could check to see if all input was consumed to determine if an error occurred.

- Tom noted that these signatures don't include an `mbstate_t` parameter and therefore cannot support processing text one byte at a time as might be required to advance through buffer boundaries; for the `mbstate_t` taking variants, all bytes could be consumed without ensuring that an error condition is not present.
- Tom suggested that optional parameters could be used to return counts of processed code units.
- Corentin expressed a preference for the design in the paper for usability reasons and noted a dislike for using a return value to indicate both errors and counts.
- Corentin stated that performance concerns should be subject to benchmarks indicating a problem, and that he is reluctant to compromise usability for small gains.
- JeanHeyd noted that the design in the paper avoids using the return value for multiple purposes.
- Zach wondered if the challenge with this interface is trying to do too many things and violating the single purpose guideline; perhaps it would be worth having separate interfaces for validation and counting.
- Zach added that examples of real world code would help to guide the design.
- Zach asked if the interface offered a mechanism to request replacement characters for ill-formed input.
- JeanHeyd replied that there are multiple possibilities for handling errors, but that WG14 felt the best approach is to leave error handling policy to the programmer.
- JeanHeyd added that these interfaces are intended to be basic low level functionality and that more complex functionality can be added at a higher level.
- Zach acknowledged that simplified wrappers could provide higher level error handling.
- Jens asserted that interface choice should be based on informed performance behaviors and acknowledged that a complicated return value is undesirable.
- Jens added that many C interfaces feel awkward from a C++ perspective due to the use of pointers and the need for manual error checking, and suggested that the focus be on getting the interface working and functional; ergonomics can be secondary.
- Jens requested that the paper reflect opinions regarding separate validation and counting functions.
- JeanHeyd stated that he would update the paper and bring back a revision for further review.
- Corentin asked JeanHeyd what additional feedback he would like, what feedback WG14 might appreciate, and whether we should expect WG14 to consider our feedback.
- JeanHeyd replied that WG14 will take WG21 feedback seriously.
- JeanHeyd stated that the feedback provided so far has been useful; examples of usage will be a good addition to the paper.
- JeanHeyd added that he'll take any questions or advice that he can't answer himself to WG14.

    - JeanHeyd noted that WG14 is open to adding potentially many functions, so separate interfaces could be a possibility.
    - Corentin expressed confidence that whatever interface is decided on will likely be reasonably usable and useful for C++, but that a dependency on WG14 is not required since WG21 can specify its own interfaces.
    - JeanHeyd agreed and noted that part of his intent in working with WG14 is to ensure that WG21 implementors have the tools they need to implement future WG21 libraries.
    - Tom asked if the intent is that these interfaces be efficiently implementable using existing interfaces like `iconv` and `MultiByteToWideChar`.
    - JeanHeyd replied, yes.
    - Tom noted that optimizing these interfaces to work with little or no overhead over those interfaces should be a goal.
    - JeanHeyd agreed.
    - Tom asked how to seek to the next valid lead code unit when an error is encountered.
    - JeanHeyd replied that the simplest solution is to increment the input by one byte and to try again.
    - Tom observed that doing so would result in many error actions taken for a contiguous sequence of ill-formed code units, e.g., issuing many replacement characters; that conforms with Unicode, but Unicode guidance is to substitute a single replacement character for such sequences.
    - JeanHeyd replied that the caller could track consecutive errors and wait to apply an error policy until the ill-formed sequence has been fully consumed.
    - Tom agreed that would work and noted that we don't need to optimize for the error case.
- P2093R3: Formatted output:
    - Victor presented:
        - Victor's presentation slides are avilable here.
        - Feedback provided during the last SG16 review was incorporated:
            - Additional motivation for `stdout` as the default output stream was added.
            - Appendix A was added with a comparison of behavior in other languages.
            - Other fixes and minor changes.
        - Six languages have been evaluated on Linux, macOS, and Windows.
            - The interesting case is Windows due to its use of a console encoding distinct from the system encoding.
            - Go, JavaScript, Python, and Rust were all able to produce correctly rendered output to the Windows console; C and Java did not.
            - When output was redirected to a file, Java and Python both tried to transcode to Windows-1251.
                - Java substituted replacement characters for unrepresentable characters.

            - Python threw an exception when attempting to convert unrepresentable characters.
        - The paper proposes following the behavior exhibited by C, Go, JavaScript, and Rust.
        - The only difference compared to `printf()` is special handling when writing to the console on Windows.
    - PBrett noted that, for Java, JavaScript, Python, and Rust, the internal encoding is always Unicode, but that is not the case for C and C++; this may indicate different behavior is warranted.
    - Victor agreed that there is a difference, but the proposal is to only special case writing to the Windows console when the execution encoding is UTF-8.
    - Corentin commented that, when writing to the console, the output is necessarily text, but when writing to a file, the output could be binary.
    - Jens noted how JeanHeyd's transcoding facilities could be used here; we can transcode so long as a target encoding is known.
    - Jens reiterated his desire for lower level functionality to be exposed such that general programmers would be able to implement similar functionality. For example:
        - A facility to determine if output will be directed to a console; Tom showed how that can be done on several platforms and it is common for such customization to be done on POSIX systems for paging and color highlighting purposes.
        - Facilities to enable transcoding. P1885 enables identifying the execution character set; the remaining missing functionality is how to write directly to the console on Windows.
    - PBrett noted that we've discussed the desire to expose the low level functionality before.
    - Jens indicated that he is strongly opposed to forwarding this paper on without those independent lower level facilities.
    - Victor responded that he is willing to propose those lower level interfaces, but would prefer to focus on the text issues within SG16; the other facilities can be LEWG concerns.
    - Tom stated that functionality to write directly to a console is arguably an SG16 concern; though also arguably an SG13 concern.
    - Jens expressed concern about writing to the console being library black magic; he would like to be able to perform such writes without having to use formatting facilities, if desired.
    - Steve suggested that use of the Windows console is becoming more rare; software developers are probably the biggest users of it.
    - Steve asserted that the don't pay for what you aren't using principle applies; transcoding overhead may be undesirable, especially if it corrupts output.
    - Victor agreed and stated that he doesn't want to break anything; he just wants to do what `printf()` does with the one exception of writing directly to the Windows console in order to avoid Windows' builtin mojibake.

    - Steve suggested that direct writing could be considered QoI; the standard should not be specifically concerned with the Windows console.
    - Steve rephrased; we want to give Windows implementors permission, not a mandate.
    - Tom raised the question from the agenda email asking about future direction to integrate support for other character types.
    - Victor replied that it is straight forward for `char16_t` and `char32_t` since they can be simply transcoded to UTF-8, but that support for `wchar_t` is more complicated due to the need to actually transcode.
    - Tom replied that it isn't straight forward what the behavior should be when the execution character set is not UTF-8; consider EBCDIC.
    - PBrett opined that we don't have to solve this issue now.
    - Tom agreed, but expressed concern that design decisions made now might result in missed opportunities to do better than `printf()` in the future.
    - Zach commented that he does not want this paper delayed while awaiting proposals for the lower level interfaces encouraged by Jens.
    - Corentin stated that LEWG is busy and urgency is required if the proposal is to make C++23.
    - Corentin added that more features can be added later and that many desired features are not Unicode concerns.
    - Hubert claimed that the usefulness of a simple interface is nice and that the special cases needed here are not so disimilar to those required for `std::filesystem`.
    - Hubert observed that the proposal is both over specified and under specified; it is over specified for Windows, but under specified otherwise.
    - Hubert noted that the literal encoding and the locale encoding are distinct, and although P1885 would allow differentiating them, we still have not standardized (in C or C++) facilities to transcode from the literal encoding.
    - **Poll: Forward P2093R3 to LEWG.**
        - **Attendance: 9 (Hubert was present for discussion, but was not able to be present for the poll)**
        - **SF F N A SA**
          4 2 2 0 1
        - **Consensus is in favor.**
        - SA stated opposition to progression without the lower level facilities also being made available.
        - Victor asked if it would still be helpful to propose those facilities separately.
        - SA responded, yes.
- Tom stated that the next telecon will be on February 24th and that the tentative topics are Jens' P2314 and discussion of priorities for C++23.

February 24th, 2021

Draft agenda:

- P2314R0: Character sets and encodings
- P2297R0: Wording improvements for encodings and character sets

Attendees:

- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Tom provided an introduction.
  - P2314R0 and P2297R0 overlap and compete in several ways.
  - The authors have been communicating with each other offline to ensure that their respective positions and the differences between the papers are understood.
  - Since the scope of P2314R0 is smaller than P2297R0, Jens will present first, we'll keep discussion to clarifying questions, then Corentin will present, then we'll open the floor to general discussion.
- P2314R0: Character sets and encodings:
  - Jens presented.
    - C++ support for source code with characters outside the basic source character set uses model A from the "UCN models" subsection of section 5.2.1 in the C99 rationale document.
    - In model A, characters not in the basic source character set are implicitly converted to *universal-character-names* (UCNs).
    - The (revised) paper proposes switching to a modified model C in which characters are converted to an internal encoding that is able to represent all supported characters as well as UCNs.
    - The model A approach is not reflective of how compilers work in practice.
    - The ISO requires that, when an ISO standard exists and suffices for a particular purpose, that it be used. As a result, the proposed wording references ISO 10646 rather than the Unicode standard.
    - The terminology used in ISO 10646 differs from the terminology in the Unicode standard.

- The C++ standard does not contain a normative reference to the Unicode standard today.
- The standard conflates character sets and character encodings and this makes defining multibyte encodings problematic.
- The wording is intended to separate the ideas of character sets vs character encoding.
- Character sets are not mentioned except where normatively needed; their existence can be otherwise inferred from encodings.
- The wording introduces a new *literal encoding* term.
- The literal encoding may not be compatible with the locale dependent execution encoding; this is consistent with the status quo.
- An earlier revision of the paper removed *execution character set* in proposed wording, but the term was revived for C compatibility.
- Not all string literals denote the same thing; some denote an object, but others are used only at translation time. For example, header names, `extern "C"`, and `_Pragma` directives.
- Terminology changes:
  - *Basic source character set* is renamed to *basic character set*.
  - *Basic literal character set* describes aditional characters that are required to be representable in all literal encodings.
  - *Ordinary literal encoding* specifies the encoding used to encode ordinary string literal objects.
  - *Wide literal encoding* specifies the encoding used to encode wide string literal objects.
  - *Translation character set* specifies the set of abstract characters corresponding to all possible characters encountered during translation; these map to UCS scalar values.
  - *Extended character set* is removed.
- The wording avoids the notion of a character being dependent on what Unicode version is used.
- There are no intended behavioral changes other than one case involving stringizing extended characters; in that case, the standard currently specifies that a UCN is stringized, but that doesn't match existing behavior.
- During translation phase 1, source characters are mapped to the translation character set.
- In translation phase 3, UCNs outside of a header name or literal are converted to the translation character set.
- There are a few questions regarding header names.
  - Implementations appear to replace UCNs in header names.
  - Extended characters don't work portably in header names.
- Translation phases 5 and 6 could be collapsed, but are preserved to avoid renumbering translation phase 7.

        - The *translation character set* members include both named characters and abstract characters for unassigned UCS scalar values.
        - The *basic character set* is defined in terms of UCS scalar values and names.
        - The *basic literal character set* consists of the *basic character set* plus characters for alert, backspace, carriage return, and null.
        - Questions remain regarding the relationship between the literal encodings and the locale dependent execution encodings.
        - The C standard wording appears to require that characters used in a character or string literal must correspond to their encoding in the locale dependent execution encoding.
    - PBrett asked if the standard would be more clear if it referred to the Unicode standard instead of ISO 10646.
    - Hubert replied that the editorial standards that the ISO imposes is salient for use in C++ and that the Unicode method of terminology doesn't meet those standards.
    - Jens added that when we reviewed Unicode terminology a while back, we didn't find the Unicode terms very palatable.
    - Jens noted that references to the Unicode standards will be required at some point to satisfy dependencies that are not present in ISO standards.
    - PBrett observed that we don't have a guarantee that requirements don't change with new Unicode standards because of our planned dependence on UAX #31.
    - Jens agreed.
    - PBrett commented that the noted behavioral change regarding stringizing a UCN appears to be a bug fix and standardizes existing behavior.
    - Hubert expressed concern that the new wording for *basic literal character set* may strengthen requirements to prohibit the same code unit value from meaning different things when it appears as a lead byte vs a trail byte.
    - PBrett stated that the core language is not dependent on locale, so discussion of execution character sets should be moved to library wording.
- P2297R0: Wording improvements for encodings and character sets:
    - Corentin presented:
        - Corentin's presentation slides are available here.
        - Jens' paper is good, but there are some areas of disagreement.
        - We agree on removing implicit production of UCNs in translation phase 1.
        - We should strive to better use terminology compatibile with Unicode.
        - It isn't clear that we all have the same mental model of translation.
        - There is disagreement with regard to Jens' *translation character set* and that translation is not expressed using Unicode terminology.
        - Translation can be defined in terms of UCS scalar values; characters are not necessary.
        - An abstract translation character set solves a problem that doesn't exist if the right terminology is used.
        - The output of translation phase 1 should be a sequence of UCS scalar values.

        - UCS scalar value may not a great term; we can use an alias, but it should not denote a character or abstract character.
        - We agree on renaming *basic source character set* to *basic character set*.
        - We disagree on the need for *basic literal character set*; we should pursue other means of handling the special requirements for alert, backspace, carriage return, and null.
        - The relationship between the literal encodings and locale dependent execution encodings have interesting ramifications:
            - `isalpha('a') // Can this ever return false?`
            - `isalpha('é') // Can this ever return false?`
        - It is ok to retain the status quo, but execution encoding concerns should be moved to library wording.
        - Raw string literal delimiters are restricted to the basic character set; perhaps that should be changed.
    - Hubert stated that EBCDIC code pages can't meet a requirement that all members of the basic character set are encoded the same in all supported locale encodings.
    - Jens noted the implication that we cannot even rely on a correspondence between the literal encoding and the execution encoding even for members of the basic character set.
    - Hubert stated that reducing the restrictions for raw literal delimiters would complicate tokenization.
    - Jens expressed a desire to poll a preference for abstract characters vs UCS scalar values.
    - Mark stated that header names effectively need to specify a sequence of bytes and it isn't clear that these should be called characters.
    - Steve noted that there is plenty of implementation-defined behavior involved in processing header names.
    - Jens stated that, with the current phrasing, a lone surrogate code point cannot be represented in a program since they are rejected as UCNs; but an implementation could allow that as an extension.
    - Jens opined that we should allow extended characters in header names.
    - Corentin noted that the status quo is maintained by both papers.
- Tom stated that the next telecon will be on March 10th and will continue this discussion.

March 10th, 2021

Draft agenda:

- Continue discussion from the last telecon with updated draft paper revisions:
    - D2314R1: Character sets and encodings
    - D2297R1: Wording improvements for encodings and character sets

Attendees:

- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- D2314R1 vs D2297R1:
  - Peter provided an introduction.
  - Corentin initiated discussion:
    - The primary disagreement concerns the introduction of "translation character set" instead of using Unicode terminology directly.
    - The proposed wording uses "abstract character" in a way that is explicitly prohibited by the Unicode standard.
    - *[Editor's note: ISO 10646 does not define "abstract character"; it is only defined by the Unicode standard.]*
    - *[Editor's note: Unicode 13, chapter 3.2, "Conformance Requirements", conformance clause C2 states:*

      > C2 A process shall not interpret a noncharacter code point as an abstract character.
      >
      > - The noncharacter code points may be used internally, such as for sentinel values or delimiters, but should not be exchanged publicly.

      ]
    - *[Editor's note: Later revisions of D2314R1 replaced "abstract character" with "character" in the proposed wording.]*
    - We should prefer well known terminology and, in this case, not divert from the Unicode standard.
    - We should avoid using ambiguous terminology like "character".
    - The technically correct term is UCS scalar value.
    - The distinction between a character and a scalar value is not relevant for a lexer; lexing can be described using either form.
    - Thinking of lexing in terms of code units or scalar values may feel unintuitive, but that reflects how implementations actually work.
  - Hubert stated that his impression of Jens' intent in using character terminology is that source code is considered text.

- Hubert continued; we tend to think of text as a sequence of characters, so it is not clear if a sequence of UCS scalar values constitutes text.
- PBrett noted prior discussions that questioned whether C++ source code constitutes text since preservation of unassigned UCS scalar values, which do not correspond to characters, is required.
- PBrett stated that the desired model is one that describes translation in terms of UCS scalar values.
- Corentin agreed with Pter.
- Steve also agreed with Peter and added that even a pedantic reading of lexing in the C++ standard should not depend on Unicode version.
- Jens agreed with regard to avoiding dependence on Unicode version with the exception of recognizing identifiers since that requires UAX #31; translation must not otherwise be affected by Unicode version.
- Jens disagreed that it is otherwise beneficial to expose UCS scalar values more widely in the standard.
- Jens acknowledged the perspective that, due to the possibility of unassigned scalar values, C++ source may not be text, but opined that such cases will largely be introduced by *universal-character-name*s (UCNs).
- Jens added that concerns regarding unassigned scalar values can be addressed in a foot note.
- PBrett asked if there is a technical concern that motivates introducing "translation character set".
- Jens replied that he did not think there is a technical requirement and acknowledged that translation could be described in the standard using UCS scalar values.
- PBrett asked if Jens considers *universal-character-name* an unattractive term and, if not, why UCS scalar value should be disfavored.
- Jens replied that UCNs are used for a limited purpose and do not appear frequently in the standard; use of UCS scalar value for translation would require many more uses of that term..
- Jens added that UCS scalar values denote an integer value and that is inconsistent with how lexing is described elsewhere.
- Jens noted an example; proliferation of UCS scalar value leads to specifying "reinterpret_cast" as a sequence of numberic values.
- Jens concluded that this debate concerns a presentation issue in the C++ standard.
- Tom noted that it is not possible to distinguish between assigned and unassigned UCS scalar values without consulting a specific version of the Unicode standard.
- Tom added that there are cases where a single abstract character is mapped to more than one UCS scalar value but where the standard must define behavior based on the UCS scalar value, not the character.
- *[Editor's note: Examples include compatibility characters such as the angstrom character (Å, mapped to U+00C5 and U+212B) and the ohm character (Ω, mapped to U+03A9 and U+2126).]*

- Corentin summarized some of the concerns; Tom and Jens are concerned about describing translation in terms of UCS scalar values because those denote an integer.
- Zach expressed uncertainty with regard to how use of UCS scalar value leads to having to describe lexing in terms of integers; implementations always represent characters using numbers.
- Tom replied that source code read from a napkin or a blackboard doesn't go through a numeric translation; we think about lexing in terms of characters, not numbers.
- Hubert stated that the goal is to require a mapping for translation without having to explicitly specify it.
- Hubert added that there is less friction if UCS scalar values are used; that means that improving presentation in the standard should be the goal.
- Zach asked to clarify how presentation would be confusing.
- Hubert replied that translation is described assuming textual representation in the standard.
- Zach asked if that could be addressed in some front matter text.
- Hubert replied that he thinks it could be.
- Corentin asked Jens if Hubert's suggestion would alleviate his concerns.
- Hubert stated that, in terms of front matter, context matters; what would be needed is to state that the glyphs used in the specification are a proxy for UCS scalar values.
- Zach suggested including such a statement with the description of the basic character set.
- Jens replied that the basic character set concerns character sets rather than tokens and lexing.
- Jens suggested that [lex.pptoken]p2 may be a more appropriate location.
- Jens acknowledged that there should be a blanket statement somewhere noting how the glyphs used in the standard are to be interpreted.
- Tom returned discussion to Corentin's question regarding whether Hubert's suggestion would alleviate his concerns.
- Jens replied that he finds it to be a useful clarification, but that he continues to believe that general readers are better served by use of an abstraction.
- Jens added that use of UCS scalar value raises the question of assigned vs unassigned characters where we don't want to make a distinction and just state that some character is denoted here.
- Jens again acknowledged that this is purely a presentation concern, not a semantic one.
- Corentin stated that use of UCS scalar value may cause confusion for readers not familiar with the term, but noted that is exactly why character is not a good term; people have different ideas of what a character is, so use of UCS scalar value will force such readers to look up the definition in order to understand the intent.
- PBrett noted that he knows of colleagues that think of code point as a character.
- Hubert stated that his impression of Jens' perspective is that we don't want people paying attention to something that might be distracting if we can gloss over it.

- Hubert expressed support for Corentin's perspective, implementors can't afford to gloss over such specification and must deep dive to determine intended behavior; UCS scalar value may be simpler.
- Jens replied that he defined translation character set as precisely as he could.
- Jens provided [lex.pptoken]p2 as an example where there is mention of white-space characters that would require replacement written in terms of UCS scalar values, but where we do not have a specification.
- Jens noted that he has been replacing use of colloquial names of characters with ISO 10646 U+XXXX short names.
- PBrett suggested this paragraph requires an update regardless then.
- Jens agreed and added that a definition of white-space should precede it.
- Jens provided [lex.pptoken]p3, as an additional example where there are many uses of "character".
- PBrett asked to clarify that the concern is the many pre-exising uses of "character" in these clauses.
- Jens replied that migrating away from "character" in this section would damage presentation.
- Jens added that it feels like a category error to say, "if the next three UCS scalar values are ..."
- Zach asked if colloquial terms could continue to be used with the previously suggested front matter.
- Jens replied that doing so is ok normatively, but still feels like a category error.
- Zach noted that in mathematical descriptions, an alternate notation is sometimes used because it is thought to be more convenient.
- Zach noted that the proposed wording doesn't replace newline.
- Jens replied that newline is special since it does not necessarily denote `\n`, it could be `\r\n`.
- Tom added that it could also represent the end of a record in a z/OS data set.
- Corentin suggested that the pre-existing concerns regarding newlines not be addressed in this paper.
- Jens agreed and stated that he did not intend to address those.
- Zach expressed a preference for retaining glyphs rather than swapping them out for U+XXXX short names and noted that "U+005C REVERSE SOLIDUS" is unambiguously less clear than "backslash `\`".
- Tom suggested that both could be presented; "U+005C REVERSE SOLIDUS (\)".
- Corentin agreed that "REVERSE SOLIDUS" is not a great name, but that including the U+XXXX short name removes ambiguity.
- Steve expressed sympathy for Zach's point; people don't read this part of the standard because they think they know what it means and then end up surprised; you have to know what it means before you can understand the wording.
- Steve agreed that, in terms of presentation, and as is seen in other standards, the redundancy of glyph, the U+XXXX short name, and the full name is helpful; if the

glyph is known, the names can be skipped and if the glyph is unknown or ambiguous, then the names unambiguously identify the character.
- Tom noted that we've discussed this question for an hour and suggested we poll it and then proceed to another topic.
- Mark asked Jens if his concerns have been addressed.
- Jens replied that he is still concerned about presentation.
- Tom wondered if we might get inspiration from other language standards that describe lexing in Unicode terms.
- PBrett responded that they all describe it in terms of characters, but that they are able to define character in a reasonable way.
- Zach suggested adding a definition of "character".
- Jens indicated no intention of renaming "character literal" and noted that, outside of lexing, translation is only concerned with tokens.
- Zach asked what the ramifications would be if core language used "character" differently than library.
- Jens opined that defining "character" and then not using it consistently would be worse than the status quo.
- Hubert stated that defining "character" to suit this specific purpose is not a good idea and that it would be a drafting violation to use an inconsistent definition.
- Steve observed that use of "character" in the library is equally as bad as in the core language.
- **Poll: Introduce the concept of a 'translation character set' which synthesizes characters for unassigned UCS scalar values.**
  - **Attendance: 9**
  - **SF F N A SA**
    1 4 0 2 2
  - **No consensus.**
- Corentin asked for clarification regarding the desire for "translation character set"; specifically as to whether the term is useful or because UCS scalar value is an unattractive term.
- Mark replied that "translation character set" allows existing implementations to remain conforming.
- Hubert responded that both approaches have no impact on conformance and that this concerns a presentation issue.
- Corentin agreed with Hubert.
- Tom attempted to clarify Mark's point, that the abstraction acknowledges the possibility of multiple implementation techniques.
- Tom expressed appreciation for that level of abstraction.
- Hubert stated that whether implementations operate on scalar values or an "in-band transport" is observable; the C99 rationale was flawed.
- Hubert added that it took us a long time to recognize the C99 model A limitations.
- Tom asked what might lead people to change their vote.
- PBrett replied that the sticking point for him is the materialization of characters.

    - Jens asked if that concern is more about the name "translation character set", or the use of "character" in the definition.
    - Jens asked if substituting "element" or another abstract term for "character" would help.
    - PBrett asked for an example of another character set that doesn't contain characters.
    - Jens replied that most character sets contain something like bell that isn't really a character.
    - Corentin opined that "element" would be an improvement, but since the elements would be equivalent to UCS scalar values, makes "translation character set" an unnecessary abstraction.
    - Tom asked if there is a way to avoid UCS scalar values that represent unassigned character from coming into play.
    - Corentin replied that he didn't think that was necessary; the mechanism described in Jens' paper is correct.
    - Hubert observed that discussion so far has been concentrated on objections to the proposed abstraction and asked what might help improve the case for UCS scalar values.
    - PBrett stated that he would prefer to have this paper with the proposed wording, than to not have it at all.
    - Jens suggested a poll to forward the paper with direction that core resolve the wording concerns and noted that, between himself and Hubert, they would be able to represent both sides of the issue.
    - Tom expressed support for that idea.
    - Mark agreed and noted that it may just be necessary for more people to weigh in on the matter.
    - Corentin stated that it seems kind of unfair to put that burden on core.
    - Tom asked if Corentin would be willing to research what additional wording changes would be necessary for Jens' paper to switch to use of UCS scalar values.
    - Corentin agreed to look into it.
    - *[Editor's note: Corentin posted suggested changes to the SG16 mailing list.]*
- Tom announced that the next telecon will be help March 24th.
- Tom noted that, due to daylight savings time changes, the next telecon will start an hour later for those in North America timezones.

March 24th, 2021

Draft agenda:

- Continue discussion from the last telecon concerning:
    - D2314R2: Character sets and encodings
    - D2297R1: Wording improvements for encodings and character sets

- Discuss priorities and goals for C++23.

Attendees:

- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann

Meeting summary:

- D2314R2 vs D2297R1:
  - PBrett introduced the topic:
    - We are continuing discussion from the last telecon, but limiting discussion to new information.
    - Corentin posted an email that proposed a wording approach he hoped might bring consensus.
  - Corentin summarized the email.
  - Jens noted that the email proposed using "universal character set", but that term is not defined in ISO 10646, so a different term or a definition would be needed.
  - Jens added that a later reply suggested use of "Universal Coded Character Set" instead; that term is defined in ISO 10646 and appears to be isomorphic to the proposed "translation character set".
  - PBrett asked if there is consensus for this direction.
  - Corentin replied that there is an additional proposed change to replace use of "abstract character" with "element" in the definition of *universal-character-name* (UCN).
  - Jens replied that he had already discontinued use of "abstract character" and that the members of the "translation character set" are denoted as "elements".
  - Tom asked for confirmation that the universal coded character set includes members corresponding to unassigned code points and was informed that it does.
  - Hubert noted that ISO 10646 uses "UCS" as an abbreviation for "Universal Coded Character Set".
  - Hubert reported an issue with the ISO 10646 specification of the UCS; it doesn't actually state what the UCS elements are.
  - Hubert added that the closest it comes to stating what those elements are is in "coded character".
  - *[Editor's note: ISO 10646:2020 defines "coded character" in 3.8:*

> **coded character**
> association between a character and a code point

]

- Steve stated in chat: "code points seem to be elements of the UCS codespace. Coded characters would be elements of the UCS, but there aren't 'characters' that correspond to unassigned codepoints. IIUC."
- Jens shared ISO 10646:2020, chapter 6, "General Structure of the UCS" and noted that the description includes a "canonical form" of the UCS that *uses* the UCS codespace.
- *[Editor's note: ISO 10646:2020, chapter 6, "General Structure of the UCS" states:*

  > ...
  > The canonical form of this coded character set — the way in which it is to be conceived — uses the UCS codespace which consists of the integers from 0 to 10FFFF.
  > ...

  ]
- PBrett observed that we don't seem to have consensus that the UCS can be used as Corentin proposed.
- Hubert confirmed and stated that, if we went forward with this, CWG would likely have to change it; probably to what Jens has proposed.
- Jens reiterated that, due to ISO rules, we are required to work with ISO 10646.
- Corentin asked if Jens' latest draft still uses "character" in the definition of translation character set or if that had ben switched to "element".
- Hubert confirmed that "character" is still used and shared the definition from the latest revision.
- *[Editor's note: that definition is:*

  > The translation character set consists of the following elements:
  > - each character named by ISO/IEC 10646, as identified by its unique UCS scalar value, and
  > - a distinct character for each UCS scalar value where no named character is assigned.

  ]
- Jens stated that "character" had to be preserved for compatibility with existing wording that uses "character".
- Hubert stated that we can't expect Jens to publish a paper that glosses over inconsistencies in the use of "character".
- PBrett and Jens both agreed.
- **Poll: Introduce the concept of a 'translation character set' which synthesizes characters for unassigned UCS scalar values.**
  - **Attendance: 9**
  - **SF F N A SA**
    2 4 1 0 1

        - **Consensus is in favor.**
        - SA: The "translation character set" abstraction is unnecessary and the definition uses terminology incorrectly.
    - **Poll: Forward D2314R2 as presented on 2021-03-24 to EWG for inclusion in C++23.**
        - **Attendance: 9**
        - **SF F N A SA**
          3 5 0 0 0
        - **Consensus is in favor.**
    - Jens stated that he will include poll results, note the objection, populate the revision section, and then submit the paper for the next mailing.
    - Corentin asked if a request for escalatation to quickly progress this paper could be made to the EWG chair since other papers will depend on it.
    - Tom replied that more motivation is needed to do so since no dependent papers have been forwarded from SG16 yet.
- Priorities and goals for C++23:
    - PBrett inroduced the topic:
        - A number of items were shared in the agenda reminder email that are candidates for prioritization for C++23.
        - We'll briefly review each; those that are nominated as a target for C++23 and for which a champion is identified will be added to the prioritization poll.
    - PBindels asked if we have a discrete vision for C++23.
    - Tom replied that that question is part of the motivation for this exercise.
    - P1628: Unicode character properties:
        - Corentin stated that he does not plan to progress this paper for C++23.
        - Not nominated.
    - P1629: Standard Text Encoding:
        - JeanHeyd stated that the scope could be reduced for C++23.
        - Steve reported good experience experimenting with the reference implementation and nominated it as a strong candidate for C++23.
        - Steve added that he would want to have the support for ranges included.
        - JeanHeyd replied that range support would probably be pursued via a different paper in order to avoid impeding progress on the core components.
        - Corentin stated that this paper would be ambitious for C++23.
        - Nominated; champion is JeanHeyd.
    - P1729: Text Parsing:
        - Tom stated that he does not know Victor's plans for this paper.
        - Corentin opined that this paper isn't really an SG16 concern until encodings are involved.
        - PBrett responded that text processing in general is in the scope of SG16.
        - Not nominated; Victor was not present.
    - P1854: Source to Execution encoding conversion should not lead to loss of information:

        - Corentin reported a request to include the wide literal encoding in the paper.
        - Corentin added that we voted to make this contingent on P1855, but that paper is still making its way through LEWG.
        - Tom stated that we don't have to wait on P1855 to be accepted to make progress on this.
        - Nominated; champion is Corentin.
    - P1859: Standard terminology for execution character set encodings:
        - Steve stated that this paper has been mostly subsumed by other work.
        - PBrett asked if we should continue tracking this paper.
        - Steve replied that we should; at least until things in flight land.
        - Jens asked that the paper be reviewed to determine when/if it can be closed.
        - Not nominated.
    - P1953: Unicode Identifiers And Reflection:
        - Corentin stated that priority of this paper depends on what SG7 does for C++23.
        - Not nominated.
    - P2071: Named universal character escapes:
        - Jens noted that a paper revision is needed.
        - Tom acknowledged and reported plans to complete a revision soon.
        - Not nominated since this paper has already been forwarded out of SG16.
    - P2295: Correct UTF-8 handling during phase 1 of translation:
        - Corentin nominated.
        - Nominated; champion is Corentin.
    - Requiring wchar_t to represent all members of the execution wide character set does not match existing practice:
        - Hubert noted that this will require corresponding changes for WG14.
        - JeanHeyd stated that other work he is doing in WG14 will help with this.
        - Nominated; champions are Tom and Corentin.
    - std::to_chars/std::from_chars overloads for char8_t:
        - Tom reported that this issue was raised from someone outside the committee.
        - Corentin stated that the concept of a number is complicated in Unicode.
        - PBindels noted that there are many different numbering systems.
        - PBrett suggested that the scope of `from_chars()` should be restricted to only parsing text that could be produced by `to_chars()`.
        - Jens stated that he had not considered other numbering systems, but had considered exposing the same functionality as for `char` for `char8_t` and UTF-8; this would suffice to provide portable ASCII support.
        - Nominated; champion is Peter Bindels.
    - Publish an SG16 library design guidelines paper:
        - PBrett suggested removing this from the candidate list since it doesn't target the standard.
        - Not nominated.
    - Deprecate std::regex:

        - PBrett noted that some NBs may object to such deprecation.
        - Nominated; champions are Peter Bindels and Peter Brett.
    - Make wide multicharacter character literals ill-formed:
        - Nominated; champion is Peter Brett.
    - Improve portable ingestion of command-line arguments:
        - JeanHeyd recalled discussion of P1275 in San Diego.
        - Tom asked if anyone knew if Isabella is planning a revision.
        - JeanHeyd replied that further work is semi-dependent on his own transcoding work.
        - Tom noted that an alternate design sketch is present in the issue comments.
        - PBindels suggested a C++26 target due to lack of underlying existing functionality.
        - Not nominated.
    - Alias barriers; a replacement for the ICU hack:
        - Tom volunteered and summarized recent off-list discussion; P0593 describes a `start_lifetime_as()` function that looks suitable for this, but it wasn't formally proposed.
        - Nominated; champions are Tom and Mark.
    - Support for UTF encodings in std::format() and std::print():
        - PBrett proposed only addressing the easy cases; the ones that don't require implicit transcoding.
        - Corentin agreed and noted that Victor recently indicated intention to write such a paper.
        - Nominated; champions are Victor and Peter Brett.
    - Specify what constitutes white-space characters:
        - Tom stated that P2295 intends to address this.
        - Corentin reported that he removed that section of the paper in a yet-to-be published revision to ensure it wouldn't delay progress on the core portion of the paper.
        - PBindels offered a comparison with P1949 and noted this shouldn't be a difficult paper, though it may not be very important.
        - Hubert stated that there may be consensus issues with this since deferring to a Unicode definition of white-space character would introduce a lexing dependency on Unicode version.
        - Steve opined that this isn't terribly important until we have portable Unicode encoded source files.
        - Steve stated that it might help to clean up the specification though; P1949 may have helped by stabilizing identifiers.
        - Corentin offered to send a message to the SG16 mailing list arguing for why this shouldn't be a priority.
        - *[Editor's note: Corentin did follow up with an email.]*
        - Not nominated.
    - Specify what constitutes a new-line:

        - Not nominated for the same reasons as the previous item.
    - A portable mechanism to specify source file encoding:
        - Tom expressed intent to work on this.
        - Jens asked that Tom prioritize P2071 first.
        - Tom agreed to do so.
        - Corentin opined that this need not be a priority.
        - Nominated; champion is Tom.
    - Clarify the relationship between the literal and execution encodings:
        - Corentin proposed this additional candidate.
        - *[Editor's note: An SG16 issue was filed to track this addition.]*
        - Nominated; champions are Corentin and Jens.
    - PBrett summarized the polling method Tom described in the agenda email.
    - Tom suggested a simpler polling method; that everyone raise their hand for items that they felt most feasible and desirable for C++23.
    - *[Editor's note: the poll results shown below have been reordered such that those that received the most votes are listed first.]*
    - **Poll: C++23 priorities.**
        - **Attendance: 9**
        - **Votes Candidate**

6	P1854: Source to Execution encoding conversion should not lead to loss of information
6	P2295: Correct UTF-8 handling during phase 1 of translation
6	std::to_chars/std::from_chars overloads for char8_t
6	Make wide multicharacter character literals ill-formed
5	P1629: Standard Text Encoding
5	Requiring wchar_t to represent all members of the execution wide character set does not match existing practice
5	Support for UTF encodings in std::format() and std::print()
4	Deprecate std::regex
4	Alias barriers; a replacement for the ICU hack
4	A portable mechanism to specify source file encoding
4	Clarify the relationship between the literal and execution encodings

- Tom announced that the next telecon will be April 14th and the agenda will include P2295: Correct UTF-8 handling during phase 1 of translation.
- Tom reminded participants from Europe of the pending switch to summer time and that the next telecon will be held an hour later than this one.